# Reinforcement Learning

# Activity: Hot or Cold

# Super Mario Brothers

## Activity: A Volunteer to Play Super Mario Brothers

https://www.youtube.com/watch?v=ldXxDNjS5jw

# What is Reinforcement Learning? I

As we saw in the Super Mario Bros video, reinforcement learning enables AI to master tasks through trial and error—starting with random actions and gradually improving through experience.

How a human player may approach the game:

- Try things out and see what works.
- Press a button, see what happens.
- If you score points, great! You'll probably try that again.
- If you lose a life, you'll avoid that action next time.

That is reinforcement learning in a nutshell:

***learning by interacting with the world and figuring out what works.***

# Why Does It Matter?

Reinforcement learning powers many exciting technologies:

- Self-driving cars learning to navigate roads
- Game-playing AI that beats human champions
- Robots learning to perform complex tasks
- Personalized recommendation systems

Most importantly, it mirrors how humans and animals naturally learn many skills—not by being explicitly taught, but by interacting with the world and learning from experience.

image generator

# What is Reinforcement Learning? II

**How do babies learn to walk?**

**The agent:** Baby

**Environment:** The physical world (floor furniture, gravity)

**Actions:** Different muscle movements, shifts in weight, stepping motions

**Rewards:**

Negative: Falling down, losing balance (pain/discomfort)

Positive: Successful steps, maintaining balance, reaching

**Goal**: Maximize successful walking (minimize falls, maximize mobility)

# Key Characteristics of Reinforcement Learning

Success comes through ***trial-and-error*** learning and experimentation

Sometimes the reward comes long after the action due to ***delayed consequences***

You must figure out which actions lead to good outcomes through ***discovery-based*** approaches

At a restaurant, do you:

- Order your favorite dish or
- Order a new dish that might be even better

**The Challenge:**

**Exploitation**

**vs.**

**Exploration**

# The Core Dilemma: Exploration vs. Exploitation

**Exploitation in reinforcement learning involves sticking with actions you already know work well.** Using existing knowledge to maximize rewards by choosing safe actions with predictable outcomes, like an agent following a proven strategy to optimize immediate returns.

**Exploration involves trying new actions to discover better strategies.** Gathering information about unknown reward structures and accepting short-term suboptimal performance to build long-term knowledge.

## Finding the Right Balance

Too much exploitation

stuck in local maxima, missing better strategies

Too much exploration

wasted time on suboptimal actions

# Optimal Strategy Evolution in RL

**High Exploration**

Try many different actions to discover possibilities.

**Gradual Shift**

Increase exploitation of successful strategies.

**Mostly Exploitation**

Apply learned knowledge consistently.

*Learning naturally shifts from exploring possibilities to using what works best*

# RL vs. Other Learning Paradigms

| Supervised Learning | Unsupervised Learning | Reinforcement Learning |
|---|---|---|
| Learn from labeled examples | Find patterns in data | Learn from rewards |
| Teacher provides correct answers | No teacher | Environment provides feedback |
| Training / test phases | No labels needed | Continuous learning |
| Classification / regression | Clustering / dimensionality reduction | Sequential decision making |

# Policy

A ***policy*** is an agent's strategy for choosing actions; it decides what to do in any given situation. Formally, a policy $\boldsymbol{\pi}$ is a function that maps states to actions (or probability distributions over actions).

## Deterministic Policy

$\boldsymbol{\pi}: S \to A$ where: $S$: state space, $A$: action space

Thus, $\boldsymbol{\pi}(s)$ is the action chosen in state $s \in S$.

Example:

$$\boldsymbol{\pi}(hungry) = eat$$
$$\boldsymbol{\pi}(enemy_nearby) = attack$$

## Stochastic Policy

$\boldsymbol{\pi}: S \times \mathrm{A} \to [0,1]$

where: $\boldsymbol{\pi}(a \mid s)$: probability of taking action $a \in A$ given state $s \in S$.

$$\sum_{s \in S} \boldsymbol{\pi}(a \mid s) = 1$$

Example:

$$\boldsymbol{\pi}(attack \mid enemy_nearby) = 0.6$$
$$\boldsymbol{\pi}(defend \mid enemy_nearby) = 0.3$$
$$\boldsymbol{\pi}(flea \mid enemy_nearby) = 0.1$$

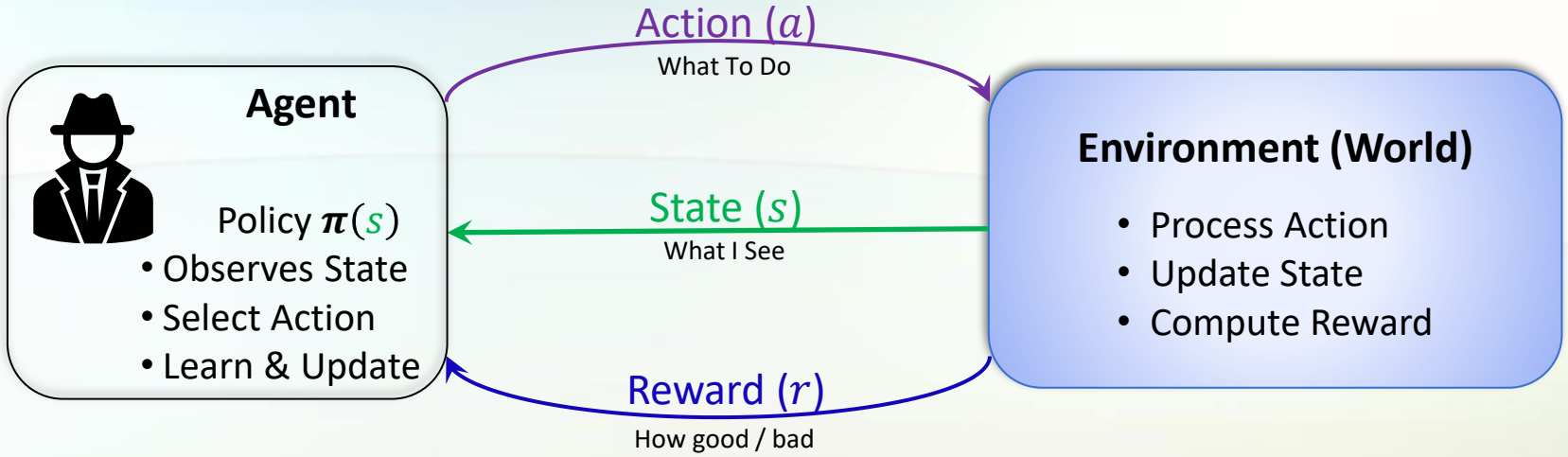
**Goal:** Find the optimal policy $\boldsymbol{\pi}^*$ that maximizes expected cumulative reward.

# The RL Setting: Agent-Environment Loop

The agent begins in an initial state $s_0$.

At each time step $t$:

$s_t \rightarrow a_t \rightarrow r_{t+1}, s_{t+1}$




**1. Action Selection**: Agent uses current policy $\boldsymbol{\pi}(s_t)$ to choose action $a_t$

**2. Environment response**: Environment transitions to new state $s_{t+1}$

**3. Learning Update:** Agent receives reward $r_{t+1}$ and improves policy for future decisions.

**Learning Goal**: Maximize the total cumulative reward $\sum r_t$

# Agent-Environment Loop Example I

Let’s trace through the agent-environment loop with a concrete example. We start with setup.

**Agent**: Robot in 3x3 grid

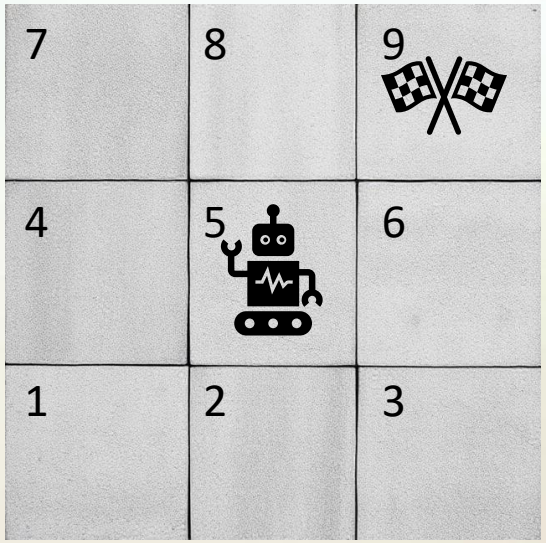
**State**: Robot's position (1-9); start in 5 (center)

**Actions**: Up, Down, Left, Right

**Goal**: Reach position 9 (top-right corner)
(we will also stipulate in 3 moves or fewer).

**Rewards**: +10 for reaching goal, -1 for each step

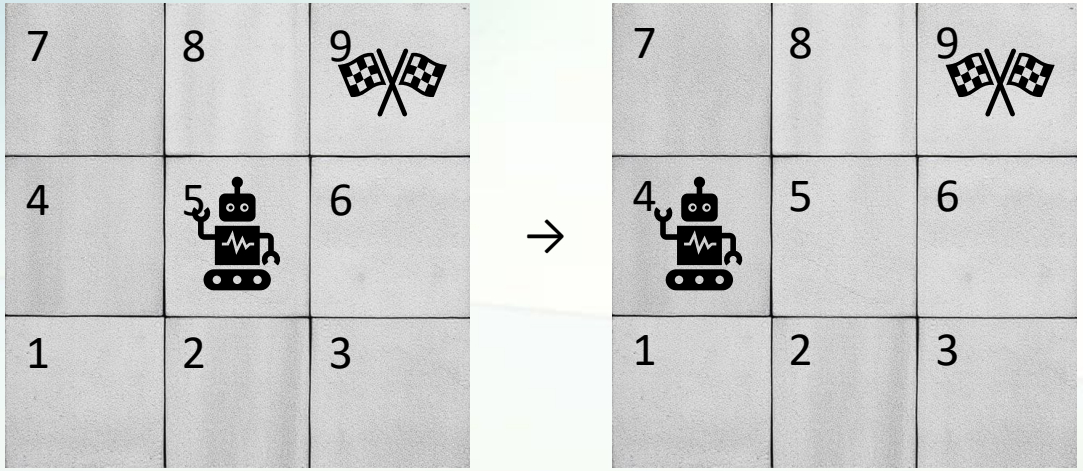
**Initial Policy**: Random (25% chance for each direction)



# Agent-Environment Loop Ex. Slide II

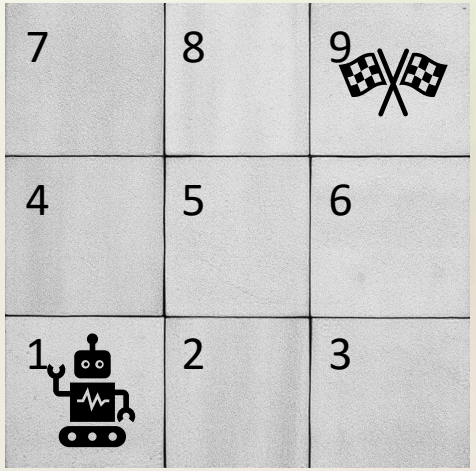
**State**: Position 5 (center)
**Policy**: $\pi(5)$ = random → chooses "Left"
**Action**: Move Left

**Environment**: Robot moves to position 4 (mid-left)
**Reward**: -1 (step penalty)
**New State**: Position 4




**State**: Position 4 (mid-left)
**Policy**: $\pi(4)$ = random → chooses "Down"
**Action**: Move Down

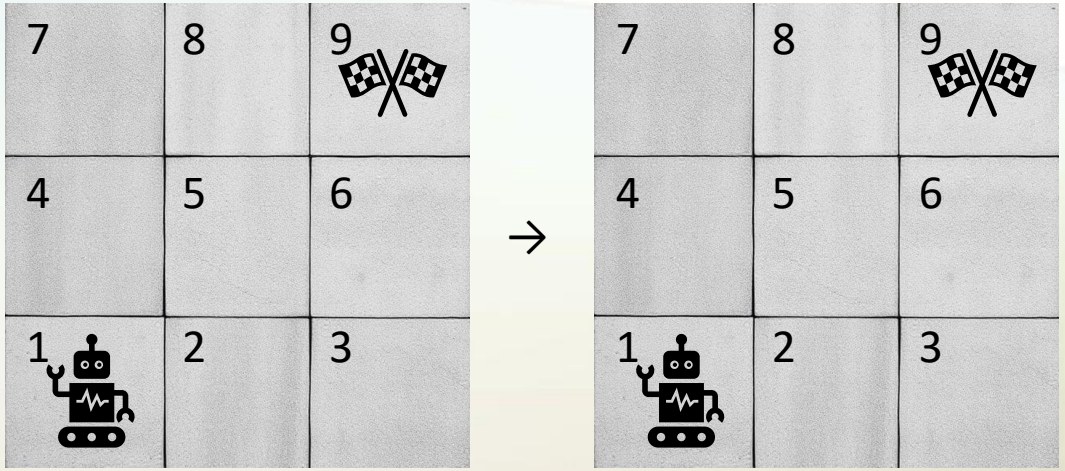
**Environment**: Robot moves to position 1 (bottom-left)
**Reward**: -1 (step penalty)
**New State**: Position 4



# Agent-Environment Loop Ex. Slide III

**State**: Position 1 (bottom-left)
**Policy**: $\pi(1)$ = random → chooses "Left"
**Action**: (Try to) Move Left

**Environment**: Hits wall, stays at 1 (bottom-left)
**Reward**: -1 (step penalty)
**New State**: Position 1: Move 3 of 3 reached.
**Episode Ends: FAILURE**



# Policy Updates (for our Example)

- A policy $\pi$ is essentially a lookup table.
- Each state has its own independent probability distribution over actions.
- Learning means adjusting these probabilities based on experience.
- The entire table IS the policy.

| State | Right | Left | Down | Up |
|---|---|---|---|---|
| 1 | 25% | 25% | 25% | 25% |
| 2 | 25% | 25% | 25% | 25% |
| … | 25% | 25% | 25% | 25% |
| 9 | 25% | 25% | 25% | 25% |

Suppose we has a policy update rule: subtract 5% from the chosen action, then renormalize.

$\pi(4)$: Down was chosen → penalize Down

| | | |
|---|---|---|
| Right: 26.32% | Right: 26.32% | Right: 26.32% |
| Left: 21.05% | Left: 26.32% | Left: 21.05% |
| Up: 26.32% | Up: 26.32% | Up: 26.32% |
| Down: 26.32% | Down: 21.05% | Down: 26.32% |

$\pi(5)$: Left was chosen → penalize Left

$\pi(1)$: Left was chosen → penalize Left

Does a table representation for $\boldsymbol{\pi}$ scale?

The $3 \times 3$ grid needs only $9 \times 4 = 36$ entries.

What about chess (~$10^{47}$ states)?

An Atari game (~$10^9$ pixel configurations)?

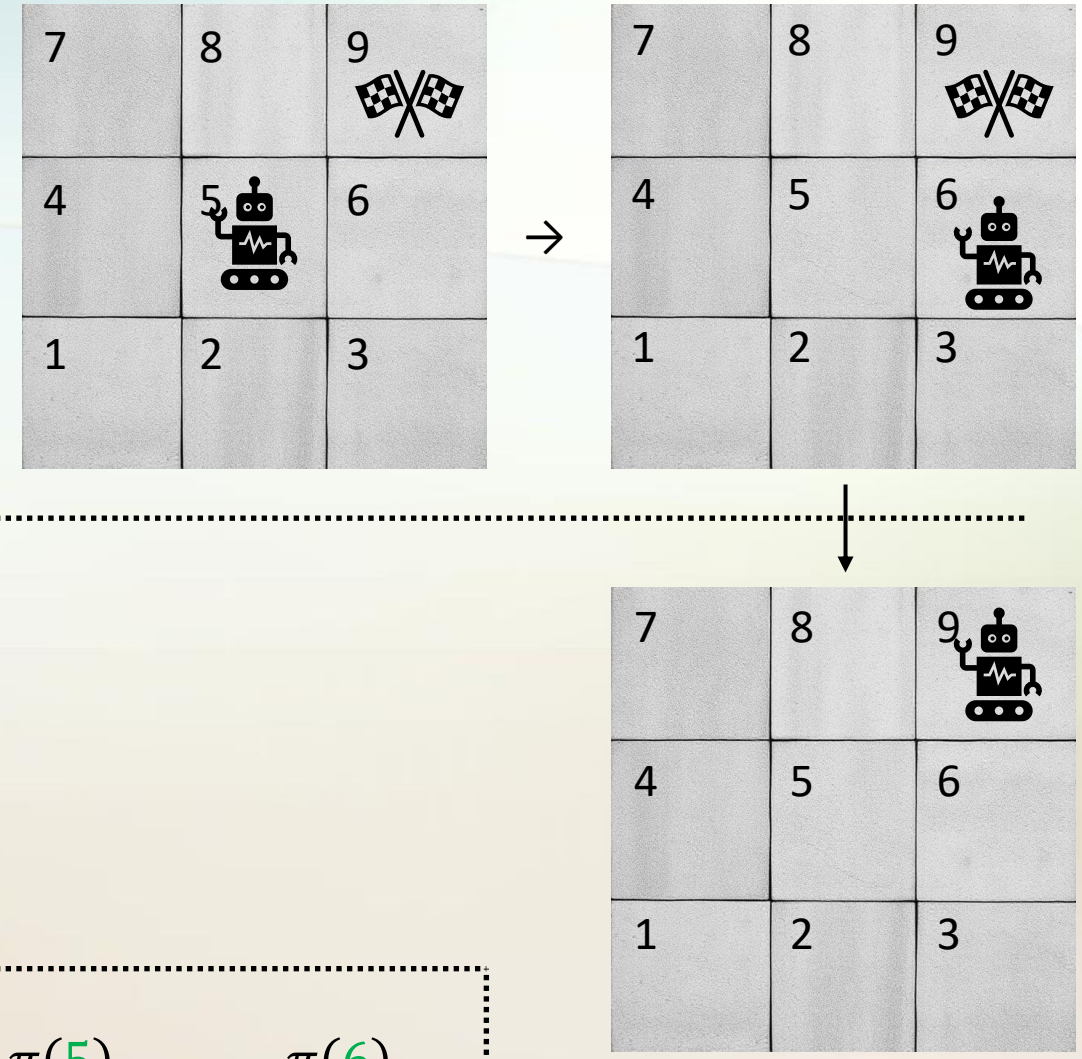
Tabular methods break down in large state spaces. (We will revisit this problem.)

# Agent-Environment Loop Ex. Slide IV

**Restart Episode: 50th Iteration**

**State**: Position 5 (center)
**Policy**: $\pi(5)$ = Learned 80% "Right" → Move Right
**Action**: Move Right

**Environment**: Moves to Position 6 (mid-right)
**Reward**: -1 (step penalty)
**New State**: Position 6




**State**: Position 6 (mid-right)
**Policy**: $\pi(6)$ = Learned 90% "Up" → Move Up
**Action**: Move Up
**Environment**: Moves to Position 9 (upper-right)
**Reward**: +10 (GOAL)
**New State**: **Episode Ends: SUCCESS**

**Policy Update**

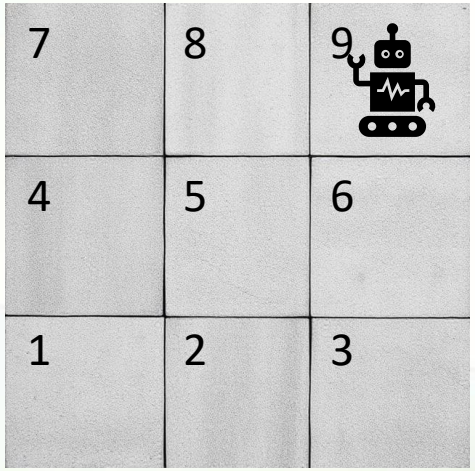
(Increase 'Good Move' directions; normalization of others omitted.)

| $\pi(5)$ | $\pi(6)$ |
|---|---|
| Right: 85% | Up: 95% |

# Agent-Environment Loop Ex. Slide V

**Final Learned Policy: $\pi^*$**

| $\pi(5)$ | $\pi(6)$ | $\pi(8)$ |
|---|---|---|
| Right: 85% | Right: 1% | Right: 90% |
| Left: 5% | Left: 2% | Left: 4% |
| Up: 5% | Up: 95% | Up: 2% |
| Down: 5% | Down: 2% | Down: 4% |




## Summary

**Episode 1**: Random movement, hitting walls, never reaching goal → **Episode 50**: Direct path to goal in just 2 steps

**Policy Evolution**: From random movement to 85-95% optimal moves.

The robot learned the shortest path to the goal purely through trial and error, with the policy encoding the optimal route!

# The Three Faces of a Policy

A ***policy*** is three things simultaneously:

1. ***What the agent learns.*** The policy reflects experience built through trial and error across episodes, encoding: 'In state $X$, take action Y.'
2. ***The Decision-Making Mechanism.*** The policy is the agent's decision engine. It is invoked at every step to answer: 'Given this state, what action?'
3. ***The Solution.*** The optimal policy $\boldsymbol{\pi}^*$ is the goal of RL. It answers what to do in every situation, fully solving the task.

This is what makes RL so elegant...

The learning process, the execution mechanism, and the solution are all the same thing.

The agent does not acquire knowledge about optimal behavior; it directly develops the policy that IS the optimal behavior.

# Some Real-World RL Examples

Some concrete examples where RL has been successful.

**Game Playing**

- **AlphaGo/AlphaZero**: Mastered Go, Chess, Shogi
- **Atari Games**: Human-level performance from pixels (Breakout, Pong, Space Invaders)
- **StarCraft II**: AlphaStar reached Grandmaster level
- **Dota 2**: OpenAI Five competed against professional players
- **Poker**: Libratus and Pluribus beat top human players
- **Hide and Seek**: OpenAI agents developed complex strategies

**Robotics**

- **Robot Grasping**: Learning to pick up various objects
- **Autonomous Vehicles**: Tesla, Waymo navigation systems
- **Drone Navigation**: Obstacle avoidance and path planning
- **Robotic Surgery**: Precision movements in medical procedures
- **Warehouse Automation**: Amazon's robotic fulfillment centers
- **Humanoid Robots**: Boston Dynamics Atlas parkour movements

**Business & Finance**

- **Algorithmic Trading**: High-frequency trading strategies
- **Portfolio Management**: Dynamic asset allocation
- **Pricing Optimization**: Airlines, hotels adjusting prices
- **Fraud Detection**: Credit card companies detecting anomalies
- **Supply Chain**: Inventory management and logistics
- **Advertisement**: Real-time bidding for online ads

**Healthcare & Science**

- **Drug Discovery**: Finding new pharmaceutical compounds
- **Personalized Medicine**: Treatment recommendations
- **Clinical Trial Design**: Optimizing patient recruitment
- **Protein Folding**: AlphaFold predicting 3D structures
- **Medical Imaging**: Improving diagnostic accuracy
- **Epidemic Control**: Optimizing intervention strategies

# Markov Decision Process

How do we formalize RL problems mathematically? We need a framework that captures the essential elements: states, actions, rewards, and transitions. (Again, this might seem slightly familiar.)

$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \cdots$$

A ***Markov Decision Process*** (**MDP**) is defined by:

$S$: Set of states

$A$: Set of actions

$P(s' \mid s, a)$: Transition probabilities.

$R(s, a, s')$: reward function

The Markov property: ***the future depends only on the current state, not on how you got there***. Formally,

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t).$$

**Goal**: Find an optimal policy $\boldsymbol{\pi}^*(s)$ that tells you which action to take in each state to maximize expected cumulative discounted reward over time.

# MDP Example I

**Agent**: a bot deciding whether to buy, sell, or hold a stock each day based on current inventory and market conditions.

**States**: Tuple consisting of:

```
(current-inventory, market-condition)
```

$3 \cdot 3 = 9$ possible states when restricted to owning 2 shares maximally.

**Action Types**: Three choices at each state

- Buy (acquire 1 share, if inventory $< 2$)
- Sell (sell 1 share, if inventory $> 0$)
- Hold (do nothing)

**Rewards**:

- Sell during Bullish market: +$10
- Sell during Neutral market: +$5
- Sell during Bearish market: -$2
- Buy: -$3 (transaction cost)
- Hold: $0

| | | Next Market | | |
|---|---|---|---|---|
| | | Bullish | Bearish | Neutral |
| Current market | Bullish | 0.6 | 0.3 | 0.1 |
| | Bearish | 0.6 | 0.3 | 0.1 |
| | Neutral | 0.4 | 0.3 | 0.3 |

Transition Probabilities:

$P(next - market \mid current - market)$

# MDP Example II: $\pi$

| 0 shares | 1 share | 2 shares |
|---|---|---|
| (0, Bullish) Bullish<br>Actions<br>Buy Share $-\$3$<br>Sell N/A<br>Hold $\$0$ | (1, Bullish) Bullish<br>Actions<br>Buy Share $-\$3$<br>Sell $+\$10$<br>Hold $\$0$ | (2, Bullish) Bullish<br>Actions<br>Buy Share Max<br>Sell $+\$10$<br>Hold $\$0$ |
| (0, Neutral) Neutral<br>Actions<br>Buy Share $-\$3$<br>Sell N/A<br>Hold $\$0$ | (1, Neutral) Neutral<br>Actions<br>Buy Share $-\$3$<br>Sell $+\$5$<br>Hold $\$0$ | (2, Neutral) Neutral<br>Actions<br>Buy Share MAX<br>Sell $+\$5$<br>Hold $\$0$ |
| (0, Bearish) Bearish<br>Actions<br>Buy Share $-\$3$<br>Sell N/A<br>Hold $\$0$ | (1, Bearish) Bearish<br>Actions<br>Buy Share $-\$3$<br>Sell $-\$2$<br>Hold $\$0$ | (2, Bearish) Bearish<br>Actions<br>Buy Share MAX<br>Sell $-\$2$<br>Hold $\$0$ |

This is the current policy $\pi$, but how do we learn $\pi^*$?

That is, we have our MDP formulation, but how do we find the best strategy?

First, we need to define how to measure the quality of different strategies.

# Value Functions: Intuition

Imagine you are a financial advisor evaluating different investment portfolios:

**Portfolio A**: Tech stocks, high growth potential

**Portfolio B**: Government bonds, stable but low returns

**Portfolio C**: Mixed portfolio with diversified holdings

An experienced investor might say:

- "Portfolio A has a value of $1.2M expected over 10 years"
- "Portfolio B has a value of $800K expected over 10 years"
- "Portfolio C has a value of $1.0M expected over 10 years"

The investor is estimating the ***expected future returns*** from each starting position, considering all possible market scenarios and the investment strategy they will follow.

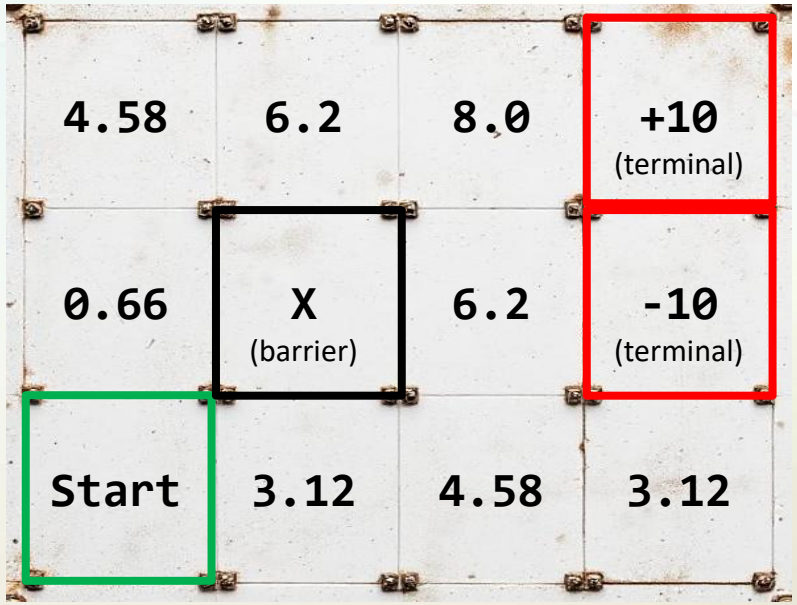
"Value function" refers to a measure of ***future reward*** potential—it is a way of estimating how promising a situation is over time.

# Value Functions: Intuition Example

"How good is each state for achieving high rewards in the future?"

We assign each state a ***value*** $V^{\boldsymbol{\pi}(s)}$: the expected cumulative reward if we start in that state and follow policy $\boldsymbol{\pi}$.

For example, $V^{\boldsymbol{\pi}(top-left)} = 4.58$ means if we start here and follow the policy, we expect to accumulate $4.58$ total reward, on average.




**Observe: higher numbers → better positions:**

- Closer to +10 terminal state (goal)
- Farther from -10 terminal state (trap)
- Fewer steps needed

* All of the these values depend on which policy $\boldsymbol{\pi}$ we are following.

# Discounting Future Rewards

Why not just sum all projected future rewards equally?

Two reasons to value immediate rewards more:

1. Uncertainty: the further out, the less certain we are we will get there
2. Opportunity: a reward received now can be reinvested

To encode this idea mathematically, we use a ***discount factor*** $\gamma$ (gamma), where $\gamma \in [0,1]$ :

| When | Reward | Discounted Value |
|---|---|---|
| Now | $r_0$ | $r_0$ |
| 1 step ahead | $r_1$ | $\gamma \cdot r_1$ |
| 2 steps ahead | $r_2$ | $\gamma^2 \cdot r_2$ |
| $n$ steps ahead | $r_n$ | $\gamma^n \cdot r_n$ |

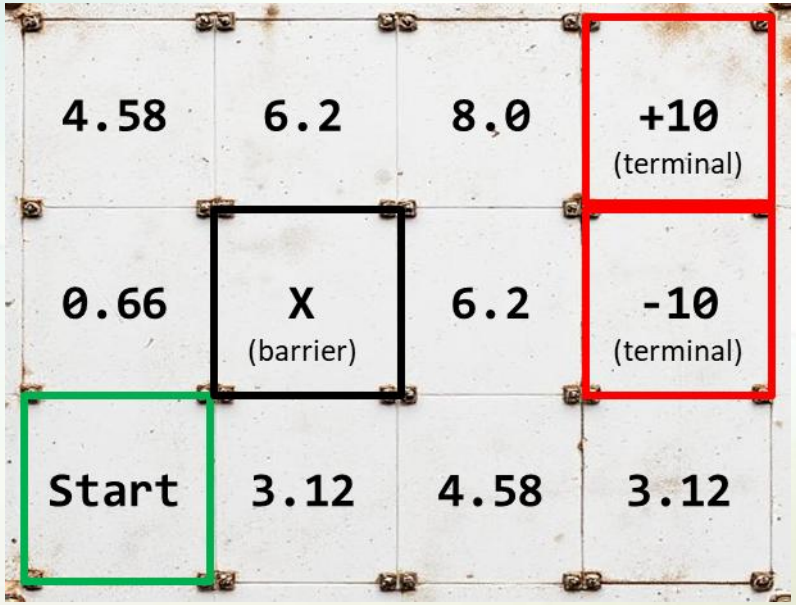
The total expected return is then given by:

$$G = r_0 + \gamma \cdot r_1 + \gamma^2 \cdot r_2 + \gamma^3 \cdot r_3 + \cdots$$

# Discounting Future Rewards (Example)

The total expected return is then given by:

$$G = r_0 + \gamma \cdot r_1 + \gamma^2 \cdot r_2 + \cdots$$




Example with our grid ($\gamma = 0.9$, $step\ cost = -1$, $goal = +10$):

One step from goal: $G = -1 + \gamma \cdot (10) = 8.0$

Two step from goal: $G = -1 + \gamma \cdot (-1) + \gamma^2 \cdot (10) = 6.2$

Three step from goal: $G = -1 + \gamma \cdot (-1) + \gamma^2 \cdot (-1) + \gamma^3 \cdot (10) = 4.58$

But the agent does not know the path.

The state value $V^{\pi(s)}$ is the expected $G$ averaged over all possible paths under policy $\boldsymbol{\pi}$.

# Recursion

We do not need to simulate infinite futures to compute value functions.

The value of any state can be broken into two parts:

**Immediate reward** we receive :

$(r)$

**Discounted value of the next state:**

$\gamma \cdot V^{\pi}(s')$

$$\boxed{V^{\pi}(s) = \mathbb{E}\left[r + \gamma \cdot V^{\pi}(s') \mid s, \pi\right]}$$

This recursive relationship is called the ***Bellman equation***, named after Richard Bellman, who developed dynamic programming in the 1950s.

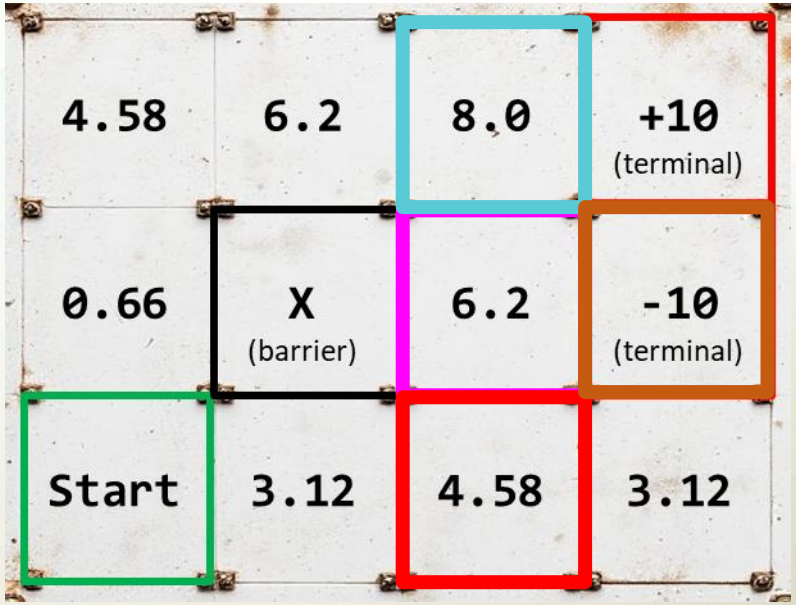
(He also developed dynamic programming in the 1950s.)

We will formalize this for policies and optimality shortly.

# Action Value Function: $Q$

$V^{\pi(s)}$ tells us how good a ***state*** is, but not which ***action*** to take.

$Q^{\pi(s,a)}$ tells us how good it is to take a specific action in a state.




Consider the middle-right containing 6.2. The ***average value*** of being here is given by: $V^{\pi(s)} = 6.2$

But which action should we take? $V$ does not tell us. $Q$ does:

$Q$-values let us compare actions. Right looks adjacent to the goal, but $Q$ reveals it is the worst possible choice.

| Action ($a$) | Next position | $Q^{\pi(s,a)}$ |
|---|---|---|
| Up | $s'$ | $-1 + 0.9 \cdot 8.0 = 6.2$ |
| Down | $s'$ | $-1 + 0.9 \cdot 4.58 = 3.122$ |
| Left | $s'$ | $-1 + 0.9 \cdot 6.2 = 4.58$ |
| Right | $s'$ | $-1 + 0.9 \cdot (-10) = -10$ |

$Q$-values are what we need to actually make decisions.

# Action Value Function: $Q^{\boldsymbol{\pi}(s,a)}$

***Definition.*** The **Bellman expectation equation for action-value function** under policy $\boldsymbol{\pi}$ gives the expected cumulative reward from taking action $a$ in state $s$, and thereafter following policy $\boldsymbol{\pi}$:

Transition probability from state $s$ to next state $s'$ after taking action $a$

Discount factor

$$Q^{\boldsymbol{\pi}(s,a)} = \sum_{s'} P(s' \mid s, a) \left[ R(s, a, s') + \gamma \cdot V^{\boldsymbol{\pi}(s')} \right]$$

Immediate reward for that transition

Expected value of the next state ($s'$) under policy $\boldsymbol{\pi}$

*Take an action, see where you might land, collect the reward, and add the discounted value of following the policy from there.*

# Connecting $V^{\pi(s)}$ , $Q^{\pi(s,a)}$, and $\pi$

State value equals the expected value of action values under the policy:

$$V^{\pi(s)} = \sum_{a \in A} \pi(a \mid s) \cdot Q^{\pi(s,a)}$$

**Intuition**:

- If we are in state $s$, our expected reward depends on what action we take
- Policy $\pi(a \mid s)$ tells us the probability of taking each action $a$
- $Q^{\pi(s,a)}$ tells us the value of each possible action
- Thus, we average all action values, weighted by how likely we are to take each action

Action value equals the expected value of immediate reward plus discounted future value :

$$Q^{\pi(s,a)} = \sum_{s' \in S} P(s' \mid s, a) \cdot \left[ R(s, a, s') + \gamma \cdot V^{\pi(s')} \right]$$

**Intuition**:

- Taking action $a$ in state $s$ gets an immediate reward $R(s, a, s')$ when we transition to some next state $s'$ with probability $P(s' \mid s, a)$
- From $s'$, we follow policy $\pi$ and receive the expected value $V^{\pi(s')}$ discounting by $\gamma$.
- Last, we compute the probability-weighted sum over all possible next states $s'$.

# Bellman Optimality Equations

What if we want the *best possible* policy, not just any policy?

The optimal policy chooses the action that maximizes expected value.

**Optimal State Value**

$$V^{*(s)} = \max_{a} \sum_{s'} P(s' \mid s, a)[R(s, a, s') + \gamma \cdot V^{*}(s')]$$

**Optimal Action Value**

$$Q^{*(s,a)} = \sum_{s'} P(s' \mid s, a)\left[R(s, a, s') + \gamma \cdot \max_{a'} Q^{*}(s', a')\right]$$

The Bellman equations give us a mathematical relationship, but we need an algorithm to actually compute the optimal values...

# Value Iteration

**Idea**: Iteratively improve value estimates using Bellman equation

**Key insight**: If we keep applying the Bellman operator, our value estimates will converge to the optimal values!

## Clarifying Notation

$\max_x f(x)$: returns the maximum value that a function achieves over the set of possible values for variable $x$.

$\operatorname{argmax}_x f(x)$: returns the argument (the specific value of $x$) that produces that maximum value.

Observe, these are both implied loops over all $x$.

```
def max_and_argmax(f, domain):
    max_val = −∞
    best_arg = null
    for each x ∈ domain:
        curr_val = f(x)
        if curr_val > max_val:
            max_val = curr_val
            best_arg = x
    return (max_val, best_arg)
```

# Value Iteration Algorithm

**Input:** MDP with states, actions, transitions, rewards, discount factor $\gamma$, threshold $\theta$

**Output:** Optimal values $V^*$, optimal policy $\boldsymbol{\pi}^*$

Set all state values to 0: $V(s) = 0$

**repeat**:

  **foreach** state $s$:

    Store the old value

    Try all actions, compute expected return for each

    Set new value ← best expected return (across all actions)

    Track how much the value changed

**until** no value changed more than $\theta$

# Extract policy:

For each state:

  Pick the action that gives the best expected return

**return** $V^*, \boldsymbol{\pi}^*$

A lot of redundant work.

# Value Iteration Algorithm

ValueIteration(MDP($S, A, P, R, \gamma$), $\theta$) → Optimal value function $V^*$, optimal policy $\boldsymbol{\pi}^*$

```
Initialize V(s) = 0  for all s ∈ S                          # value estimates
repeat:
    Δ = 0
    For each state s ∈ S:                                                    O(|S|)
        v = V(s)                                            # V_k(s)
        V(s) = max_{a∈A} Σ_{s'} P(s' | s,a) · [R(s,a,s') + γ · V(s')]   # V_{k+1}(s)   O(|A||S|)
        Δ = max(Δ, |v − V(s)|)                              # V_{k+1}(s) − V_k(s)
until Δ < θ

For each state s ∈ S:                                       # Extract policy
    π(s) = argmax_{a∈A} Σ_{s'} P(s' | s,a) · [R(s,a,s') + γ · V(s')]
return V* = V, π* = π
```

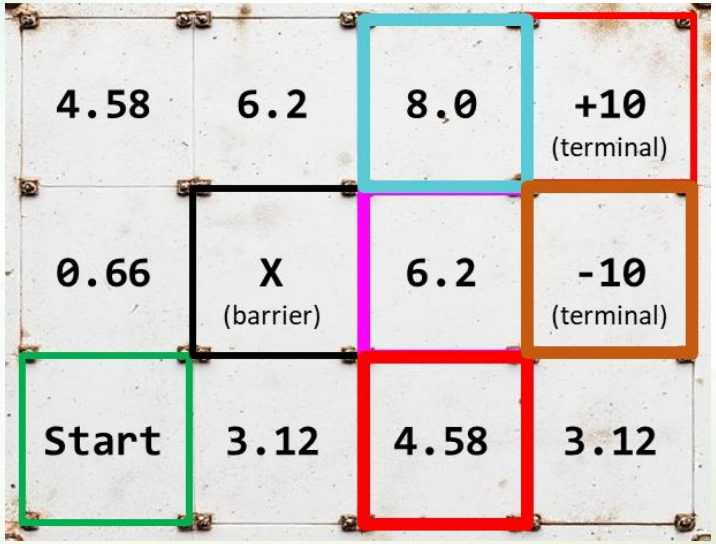
**Convergence**: Guaranteed for $0 < \gamma < 1$

**Complexity**: $O(|S|^2|A|)$ per iteration

# Revisiting the Value Iteration Example

At the 6.2 cell, we compute $Q$ for each action:

| Action ($a$) | Next position | $Q^{\pi(s,a)}$ |
|---|---|---|
| Up | $s'$ | $-1 + 0.9 \cdot 8.0 = 6.2$ |
| Down | $s'$ | $-1 + 0.9 \cdot 4.58 = 3.122$ |
| Left | $s'$ | $-1 + 0.9 \cdot 6.2 = 4.58$ |
| Right | $s'$ | $-1 + 0.9 \cdot (-10) = -10$ |




Value Iteration stores

$$V(s) = \max_a Q^{\pi(s,a)} = 6.2$$

But we discard which action achieved the max (**Up**)!
Next iteration, we recompute all four Q-values from scratch.

ValueIteration(MDP($S, A, P, R, \gamma$), $\theta$) → Optimal value function $V^*$, optimal policy $\pi^*$

```
Initialize V(s) = 0  for all s ∈ S                    # value estimates
repeat:
    Δ = 0
    For each state s ∈ S:
        v = V(s)                                      # V_k(s)
        V(s) = max_{a∈A} Σ_{s'} P(s' | s,a) · [R(s,a,s') + γ · V(s')]   # V_{k+1}(s)
        Δ = max(Δ, |v − V(s)|)                        # V_{k+1}(s) − V_k(s)
until Δ < θ

For each state s ∈ S:                                 # Extract policy
    π(s) = argmax_{a∈A} Σ_{s'} P(s' | s,a) · [R(s,a,s') + γ · V(s')]

return V* = V, π* = π
```

# Toward Policy Iteration

**The Problem ('Waste' with Value iteration's).** Each iteration, you compute the best action for every state, but then we throw away all that policy information and recompute it from scratch next time.

$$V(s) = \max_{a \in A} \sum_{s'} P(s' \mid s, a) \cdot [R(s, a, s') + \gamma \cdot V(s')]$$

This $\max_{a \in A}$ computation finds the best action, but we do not store which action that was!

**Policy Iteration* Idea**: Instead of finding optimal values first, alternate between evaluating and improving the current policy.

* Often faster than value iteration in practice

# Policy Iteration Algorithm

**Input:** MDP with states, actions, transitions, rewards, discount factor $\gamma$, threshold $\theta$

**Output:** Optimal values $V^*$, optimal policy $\boldsymbol{\pi}^*$

Start with an arbitrary policy (e.g., random action in each state)

**repeat**:

    # Policy Evaluation: How good is our current policy?

    **repeat**:

        For each state $s$, compute value assuming we follow current policy

    **until** values stop changing ($< \theta$)

    # Policy Improvement: Can we do better?

    Assume policy is stable

    **foreach** state $s$:

        Find the action with best expected return

        If this differs from current policy:

            Update the policy

            Mark policy as not stable

    **if** policy is stable:

        **return** $V^*, \boldsymbol{\pi}^*$

Only compute a score for each state (following the policy)

Same expensive update as Value Iteration, but only once per outer loop here.

# Policy Iteration Algorithm

PolicyIteration(MDP$(S, A, P, R, \gamma), \theta$) → Optimal value function $V^*$, optimal policy $\boldsymbol{\pi}^*$

```
Initialize π(s) arbitrarily for all s ∈ S
while true:
    # Policy Evaluation
    repeat:                                                                  O(k)
        Δ = 0                                                                ·
        For each state s ∈ S:                                                O(|S|)
            v = V(s)                                   # V_k(s)              ·
            V(s) = Σ_{s'} P(s' | s, π(s)) · [R(s, π(s), s') + γ · V(s')]     # V_{k+1}(s)   O(|S|)
            Δ = max(Δ, |v − V(s)|)                     # V_{k+1}(s) − V_k(s)  =
    until Δ < θ                                                              O(k|S|²)
    # Any Policy Improvement?
    For each state s ∈ S:                                                    O(|A||S|²)
        if π(s) ≠ π(s) := argmax_a Σ_{s'} P(s' | s, a) · [R(s, a, s') + γ · V(s')] then break
return V* = V, π* = π
```

# Comparing Value Iteration with Policy Iteration

| | Value Iteration | Policy Iteration |
|---|---|---|
| **Per Iteration Complexity** | $O(\|A\|\|S\|^2)$ | $O(k\|S\|^2) + O(\|A\|\|S\|^2)$ (evaluation + improvement) |
| **Convergence Speed** | More iterations, faster per iteration | Fewer iterations, slower per iteration |
| **Policy Representation** | Implicit (extracted at end) | Explicit throughout algorithm |
| **Numerical Stability** | Generally stable | Can have issues with policy evaluation convergence |
| **Implementation Complexity** | Simpler to implement | More complex due to two phases |
| **When to Use** | Large action spaces, want simplicity, memory is limited, easy parallelization is needed | Small action spaces, want faster convergence, You need good intermediate policies, You can afford the computational cost per iteration |
| **Convergence Guarantee** | Always converges to $V^*$ | Always converges to $\boldsymbol{\pi}^*$ in finite steps |
| **Intermediate Policies** | May be poor until convergence | Monotonically improving |
| **Early Stopping** | Can stop when value changes are small | Must wait for policy stability |

# When the Model is Unknown

Value and Policy iteration make one bold assumption…

that we know the Markov Decision Process model.

In particular, we do not know the transition probabilities and rewards:

$P(s' \mid s, a)$ and $R(s, a, s')$.

Where will I land? What reward will I get?

In the real world, we often do not have access to this information.
So how can we learn optimal behavior through experience alone?

Fundamentally, how do we move from a model-based learning to model-free learning?
That is, how can we learn through experience alone?

# Q-Learning

Enter $Q$-learning: instead of learning state values $V(s)$, we learn state-action values $Q(s, a)$

Instead of computing expected values, we learn from experience.

1. Take action $a$ in state $s$
2. Observe reward $r$ and new state $s'$
3. Update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\overbrace{\Big[\underbrace{r + \gamma \cdot \max_{a'}[Q(s',a')]}_{\text{target—“What we learned”}} - Q(s,a)\Big]}^{\text{error—“Difference from current estimate”}}$$

$\alpha$: learning rate

No model needed; only interaction with the environment.

# Q-Learning Algorithm ($\epsilon$-Greedy)

$\alpha$: learning rate
$\gamma$: discount factor
# episodes: tunable parameter

Initialize $Q(s,a) = 0$ for all states and actions

**repeat** (for each episode):
    start in some state $s$

    **repeat** (until episode ends):
        Choose an action $a$ ($\epsilon$-greedy: with probability $1-\epsilon$ choose the best action, with probability $\epsilon$ choose a random action)
        Take action $a$, observe reward $r$ and new state $s'$

        Update $Q$:
            $target \leftarrow r + \gamma \cdot$ (best $Q$-value from $s'$)
            $error \leftarrow target - Q(s,a)$
            $Q(s,a) = Q(s,a) + \alpha \cdot error$
        $s = s'$

$\epsilon$: exploration factor

Extract policy:
    For each state, pick the action with highest Q-value

# Q-Learning Algorithm ($\epsilon$-Greedy)

```
def Q_Learning(α, γ, ϵ, num_episodes):
    Initialize Q(s,a) = 0 for all s,a
    for episode = 1 to num_episodes:
        s = initial_state()

        while s is not terminal:
            # Choose action using ϵ-greedy policy
            if random() < ϵ:
                a = random_action()    # explore
            else:
                a = argmax_a Q(s,a)        # exploit

            # Take action and observe outcome
            s′, r = take_action(s, a)

            # Q-learning update
            Q(s,a) = Q(s,a) + α[r + γ max_a′(Q(s′, a′)) − Q(s, a)]
            s = s′

    return Q* = Q
```

$\alpha$: learning rate

$\gamma$: discount factor

$\epsilon$: exploration factor

A goal state or state with no outgoing actions

# Q-Learning Example (γ = 0.9, α = 0.5) I

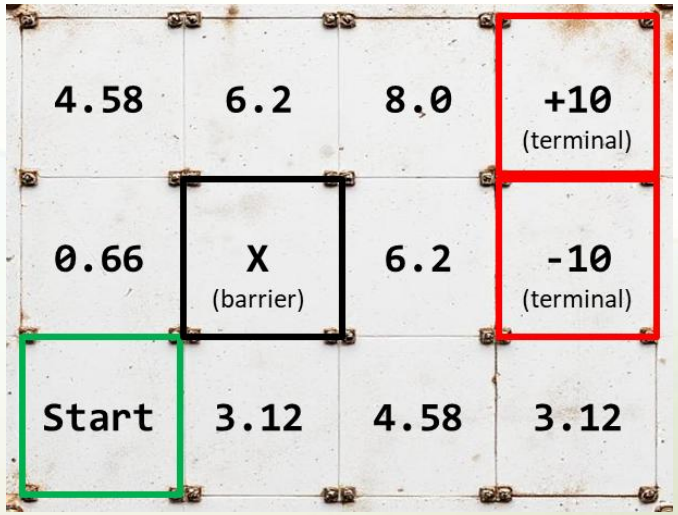
Initialize: $Q(s, a) = 0$

---

Episode 1, Step 1

**State**: Start
**Action** : Right (exploring)
**Result**: Land in 3.12 cell, reward: -1

- $target = -1 + 0.9 \cdot \max[Q(\text{3.12 cell, all actions})] = -1 + 0.9 \cdot 0 = -1$
- $error = -1 - 0 = -1$
- $Q(Start, Right) = 0 + 0.5 \cdot (-1) = -0.5$



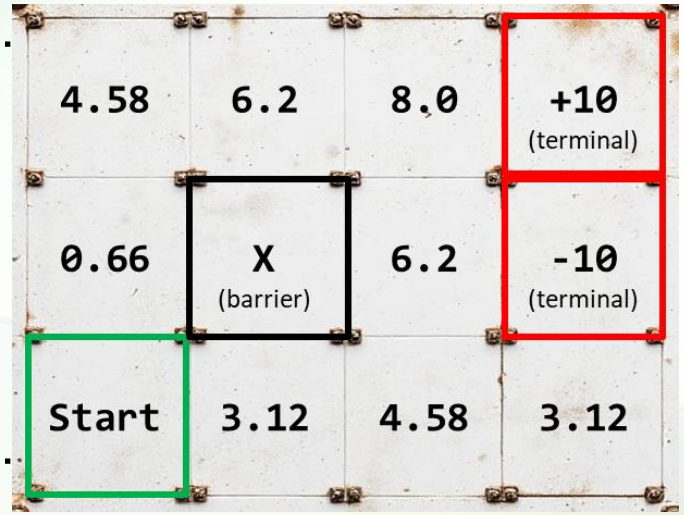

---

Episode 1, Step 2

**State**: 3.12
**Action** : Right (exploration)
**Result**: Land in 4.58 cell, reward: -1

- $target = -1 + 0.9 \cdot \max[Q(\text{6.2 cell, all actions})] = -1 + 0.9 \cdot 0 = -1$
- $error = -1 - 0 = -1$
- $Q(3.12\ cell, Right) = 0 + 0.5 \cdot (-1) = -0.5$

# Q-Learning Example ($\gamma = 0.9$, $\alpha = 0.5$) II




Episode 1, Step 3

**State**: $4.58\ cell$
**Action** : Up (exploring)
**Result**: Land in 6.2 cell, reward: -1

$$Q(4.58\ cell, Up) = 0 + 0.5 \cdot (-1) = -0.5$$

Episode 1, Step 4

**State**: $6.2\ cell$
**Action** : Up (exploring)
**Result**: Land in 8.0 cell. reward: -1

$$Q(6.2\ cell, Up) = 0 + 0.5 \cdot (-1) = -0.5$$

Episode 1, Step 5

**State**: $8.0\ cell$
**Action** : Right (exploring)
**Result**: Goal! reward: +10, EPISODE ENDS

$$Q(8.0\ cell, Right) = 0 + 0.5 \cdot (10 - 0) = +5.0$$

After Episode 1, we have:

- $Q(Start, Right) = -0.5$
- $Q(3.12\ cell, Right) = -0.5$
- $Q(4.58\ cell, Up) = -0.5$
- $Q(6.2\ cell, Up) = -0.5$
- $Q(8.0\ cell, Right) = +5.0$
- All other Q-values = 0

**After many episodes**

$Q$-values propagate backward from the goal; the agent learns which actions lead toward +10 and away from -10 without ever knowing without knowing where actions lead or what rewards they give in advance: $P(s' \mid s, a)$ or $R(s, a, s')$.